# Reproduction and Critical Evaluation of a Malware Detection Article

Final Project — Data Science in Cyber

Dr. Uri Itai

Source reproduced: github.com/emr4h/Malware-Detection-Using-Machine-Learning

# 1. Summary of the Source

## Source

"Malware Detection Using Machine Learning" — a GitHub project by emr4h (github.com/emr4h/Malware-Detection-Using-Machine-Learning), consisting of three Jupyter notebooks (KNN, Decision Tree, Random Forest) and a data-generation notebook.

## Problem being addressed

Classifying Windows Portable Executable (PE) files — i.e. .exe binaries — as malware or legitimate, using only static metadata extracted from the PE header, without executing the file or performing deep static/dynamic analysis.

## Why the problem is important

Signature-based antivirus is reliable against previously-seen malware but fails against novel or polymorphic variants whose signatures have not yet been catalogued. A lightweight classifier operating on cheap-to-extract header metadata could, in principle, provide a fast first-pass filter — flagging suspicious files for deeper analysis at a fraction of the cost of full static or dynamic analysis.

## Proposed solution

Extract 8 numeric fields from the PE header using the `pefile` Python library (`AddressOfEntryPoint`, `MajorLinkerVersion`, `MajorImageVersion`, `MajorOperatingSystemVersion`, `DllCharacteristics`, `SizeOfStackReserve`, `NumberOfSections`, `ResourceSize`), and train three standard supervised classifiers (K-Nearest Neighbors using sklearn's default k=5; source README states k=1, Decision Tree, and Random Forest) to predict a binary malware/legitimate label.

## Dataset

`MalwareDataSet.csv` — 137,444 samples, 8 numeric features, no missing values, with a binary label column named `legitimate`. Per the README, 96,526 samples are safe and 40,918 are malware (a roughly 70/30 split). Critically, by examining the source's own data-generation code, we confirmed the label is encoded inversely to its name: `legitimate = 1` actually means **malware**, and `legitimate = 0` means **safe**.

## Model / methodology

For each of the three models, the author applies a single random `train_test_split` (75% train / 25% test, no fixed random seed, no stratification), fits the model on the raw, unscaled features, and reports `f1_score(..., average='micro')` as the “success rate”: 97.6% (KNN), 98.6% (Decision Tree), 99.1% (Random Forest).

# 2. Critical Evaluation

## Main claims

That PE-header metadata alone, fed into simple off-the-shelf classifiers with no preprocessing, achieves near-perfect malware detection (97.6–99.1% “success rate”), with Random Forest the best performer.

## Are the claims supported by the evidence?

Only partially. We identified and empirically confirmed several issues that materially inflate the reported numbers:

- **Train/test leakage via unremoved duplicates.** 76% of the dataset's 137,444 rows (104,774 rows) are exact duplicate feature vectors, and the author's `train_test_split` is applied *before* any deduplication. We reproduced the author's exact pipeline and obtained results nearly identical to theirs (97.1/98.6/99.0%), then re-ran the same models on deduplicated data with a stratified split: accuracy dropped to 92.6% (KNN), 94.5% (Decision Tree), 95.7% (Random Forest), and MCC fell substantially for every model (e.g., Random Forest from 0.976 to 0.891). Full results are in Section 5.
- **The headline metric is mislabeled, not just inflated.** The author reports an “F1 Score” computed with `average='micro'`, which for binary classification is mathematically identical to plain accuracy. Despite the label, no precision/recall trade-off on the malware class is actually being measured.
- **No reproducibility control.** No `random_state` is set on the split, so re-running the notebook yields a different number each time.
- **The duplication is not random, and that changes the imbalance story.** After deduplication, the legitimate class shrinks far more (96,526 → 8,804 unique rows) than the malware class (40,918 → 23,866 unique rows) — the class balance literally flips, with malware becoming the 73% majority. This suggests many legitimate files in this corpus share near-identical header metadata (e.g., multiple DLLs from the same vendor/build), while malware samples are individually more varied. The source never addresses class imbalance in either direction.
- **Evaluation methodology.** A single hold-out split with no cross-validation and no confidence interval is treated as the final word on performance.

## Is the evaluation methodology appropriate?

No — a single, non-stratified, non-deduplicated, unseeded train/test split is not an adequate evaluation procedure for a claim this strong (near-perfect detection), especially given how sensitive the result turns out to be to the duplication issue.

## Are the conclusions justified?

Partially. The core idea — that static PE header metadata carries real signal for malware classification — does hold up under correction: even after deduplication, stratified splitting, and proper scaling, Random Forest still reaches ~95.7% accuracy and MCC ~0.89 (5-fold cross-validated: MCC 0.898 ± 0.005) on data with no leaked duplicates, which is genuinely strong for only 8 raw integer features. However, the specific claim of ~99% accuracy as a measure of real-world generalization is **not** supported — it is primarily an artifact of evaluation methodology (leakage), not model quality. Where our findings contradict the author's: the author's results materially overstate generalization performance, and the metric they label “F1” is not actually measuring what its name implies for this binary classification setting.

# 3. Feature Engineering Analysis

## Was feature engineering performed?

Essentially none. The author's notebooks slice the first 8 raw columns directly (`data.iloc[:, [0,1,2,3,4,5,6,7]]`) straight into the models — no scaling, no encoding, no transformation, no feature selection or creation.

## Features used

All 8 raw PE-header integers listed in Section 1, with wildly different scales (e.g. `AddressOfEntryPoint` up to ~1.07 billion vs. `NumberOfSections` up to 40).

## Transformations applied (and what we found by testing them)

- **Feature scaling — absent in the source, and it matters.** KNN is distance-based, so large-magnitude features can dominate Euclidean distance purely by scale, not relevance. We tested this directly: on the deduplicated data, standardizing features raised KNN's MCC from 0.708 to 0.804 (single split) and stabilized at 0.813 under 5-fold cross-validation. Tree-based models are scale-invariant, so this specifically affects KNN, which the author treats identically to the tree models.
- **Bitmask decoding — a transformation the source should have applied.** `DllCharacteristics` is not a single ordinal quantity but a 16-bit flag field per the PE/COFF spec (e.g. individual bits for ASLR support, DEP/NX compatibility, no-SEH). We decoded it into 8 individual boolean flags and found several (e.g. `nx_compat_dep`, `dyn_base_aslr`) correlate with the label as strongly as, or more interpretably than, the raw integer — "this binary does not support DEP" is a directly meaningful security signal that the raw integer value obscures.

## Is there redundancy in the system?

We checked via Spearman correlation (chosen over Pearson — see Section 5 for justification) and Variance Inflation Factor (VIF). No feature pair exceeds $|\rho| = 0.7$ (strongest: `MajorOperatingSystemVersion` vs. `SizeOfStackReserve`, $\rho = -0.68$), and all VIF scores are well under the conventional concern threshold of 5. There is a moderate cluster (`MajorLinkerVersion`, `MajorOperatingSystemVersion`, `DllCharacteristics`, `ResourceSize`) that likely reflects a shared underlying "build toolchain profile" rather than 4 independent signals — mild, not severe, redundancy. We would not drop any features on this basis; tree ensembles tolerate moderate correlation well, and the dataset is small enough (8 features) that dimensionality reduction (e.g. PCA) would cost interpretability — a real concern in a security context where analysts need to know *why* a file was flagged — for no real benefit.

## Was the feature engineering process meaningful?

The source's near-total lack of feature engineering is a real weakness, both mathematically (we showed scaling measurably helps the distance-based model) and from a security standpoint (treating a flag bitmask as an arbitrary integer discards directly interpretable signal). That said, the underlying 8 fields are a reasonable starting point: they capture *how* a binary was built/linked, which can correlate with malicious intent (e.g. malware often uses non-standard linkers, packers, or unusual section counts to evade detection) — but this is a thin, build-artifact-level signal rather than a behavioral one, and is at real risk of simply

reflecting the specific toolchains used by the samples in this particular corpus rather than a general property of malicious intent.

## Additional features that could improve performance

- Section-level **entropy** (high entropy often signals packed/encrypted payloads).
- **Imports/API calls** referenced (e.g. presence of `VirtualAlloc`, `WriteProcessMemory` is a stronger behavioral signal than header metadata).
- **Digital signature / certificate validity** (legitimate commercial software is far more often signed).
- **File size** and **section-name anomalies** (non-standard section names are a common evasion indicator).
- **Build timestamp sanity** (the PE header's `TimeDateStamp` field, not extracted by the source at all, is often zeroed or forged in malware).

# 4. Reproducibility Analysis

## Can the code be executed successfully?

Only partially. The three model notebooks (KNN, Decision Tree, Random Forest) import `plot_confusion_matrix`, `plot_precision_recall_curve`, and `plot_roc_curve` from `sklearn.metrics`, all deprecated in scikit-learn 1.0 and **removed** in 1.2+. We confirmed directly that on scikit-learn 1.8.0, all three imports raise `ImportError`. The training/evaluation logic itself (split, fit, predict, score) still runs; only the visualization cells break — but the notebooks do not execute top-to-bottom as published. Separately, `Data_Set_Generator.ipynb`, which documents how the CSV was built from raw .exe files via `pefile`, has a missing `for file in malware:` loop header (present for the `secure` folder, absent for `malware`) — as published, this cell would raise an `IndentationError`.

## Are all required files and dependencies available?

The processed CSV is included and downloads cleanly — the main reproducibility lifeline, since the modeling results can be reproduced without the raw .exe corpus. The raw malware/benign executables used to build the CSV are not included (reasonably, for safety/distribution reasons), so the data-generation step itself is not independently reproducible. There is no `requirements.txt`, `environment.yml`, or any version pin anywhere in the repository.

## Hidden preprocessing steps?

None beyond what is shown in the code — but the label semantics (`legitimate=1` means malware) are only discoverable by reading the data-generator notebook's inline comments, since the column name suggests the opposite. This is a documentation gap rather than a code bug, but it is exactly the kind of thing that causes silent errors if the dataset is reused under the assumption of conventional naming.

## Overall reproducibility

**Partial.** The dataset and the core modeling logic (split → fit → predict → score) are reproducible with minor fixes (e.g. swapping the deprecated plotting calls for `ConfusionMatrixDisplay.from_estimator`). The full pipeline from raw executables to dataset is not reproducible without sourcing an independent malware/benign corpus and fixing the missing loop. No seed is set, so the exact reported numbers are not exactly reproducible run-to-run, though our re-run lands within ~0.3–0.9 points of them, which is itself evidence we replicated the methodology correctly.

# 5. Experimental Results

## Experiments performed

All experiments were run on the actual `MalwareDataSet.csv` (137,444 rows, 8 features) downloaded directly from the source repository. We ran two parallel pipelines for every model so the leakage effect could be isolated cleanly:

- **Pipeline A (naive replication):** the author's exact methodology — split on raw, un-deduplicated data, no scaling, no stratification.

- **Pipeline B (corrected):** deduplicate first, then a stratified 75/25 split with a fixed seed, features standardized for KNN only (tree models are scale-invariant).

## Modifications introduced

- Deduplication of feature+label rows prior to splitting.
- Stratified splitting to preserve class proportions.
- Fixed random seed (42) throughout, for exact reproducibility.
- Feature standardization for the distance-based model (KNN) only.
- The source description is ambiguous regarding the KNN setting, but the executable source code uses sklearn's default KNeighborsClassifier, whose default is k=5. Therefore, the main reproduction uses KNN with k=5.
- Decoded `DllCharacteristics` into 8 individual boolean security flags (evaluated separately as a feature-engineering probe, Section 3).
- 5-fold stratified cross-validation on the deduplicated data, to obtain a variance estimate the source never reports.

## Models trained

K-Nearest Neighbors, Decision Tree, and Random Forest — matching the source's choice for direct comparability.

## Evaluation metrics used, and why

We report accuracy (for direct comparability with the source), precision/recall/F1 on the malware class specifically, Fβ (β=2, weighting recall above precision — in malware detection, missing real malware is normally costlier than a false alarm), Matthews Correlation Coefficient (MCC — remains meaningful under class imbalance, unlike accuracy or F1 alone, since it uses all four confusion-matrix cells), and ROC-AUC (threshold-independent ranking quality). We deliberately avoid the source's micro-averaged “F1”, since for binary classification it is mathematically identical to accuracy and hides exactly the precision/recall trade-off that matters most here.

## Results: naive replication vs. corrected pipeline

| Pipeline | Model | Accuracy | Precision | Recall | F1 | F2 | MCC |
|---|---|---|---|---|---|---|---|
| A: Naive (leaky) | KNN (k=5) | 0.9705 | 0.9451 | 0.9575 | 0.9513 | 0.9550 | 0.9301 |
| A: Naive (leaky) | Decision Tree | 0.9860 | 0.9797 | 0.9737 | 0.9767 | 0.9749 | 0.9667 |
| A: Naive (leaky) | Random Forest | 0.9898 | 0.9833 | 0.9826 | 0.9830 | 0.9827 | 0.9756 |
| B: Corrected | KNN (k=5) | 0.9258 | 0.9447 | 0.9542 | 0.9495 | 0.9523 | 0.8101 |
| B: Corrected | Decision Tree | 0.9450 | 0.9586 | 0.9665 | 0.9625 | 0.9649 | 0.8595 |
| B: Corrected | Random Forest | 0.9570 | 0.9699 | 0.9713 | 0.9706 | 0.9710 | 0.8907 |

Pipeline A (naive replication) reproduces the author's claimed 97.6/98.6/99.1% almost exactly (we obtain 97.1/98.6/99.0%), confirming we modeled their pipeline correctly. Pipeline B (deduplicated, stratified, scaled-where-needed) shows a clear, consistent drop across every model and metric — largest for KNN, smallest for Random Forest — directly demonstrating that a meaningful share of the source's reported performance is a leakage artifact rather than genuine generalization ability.

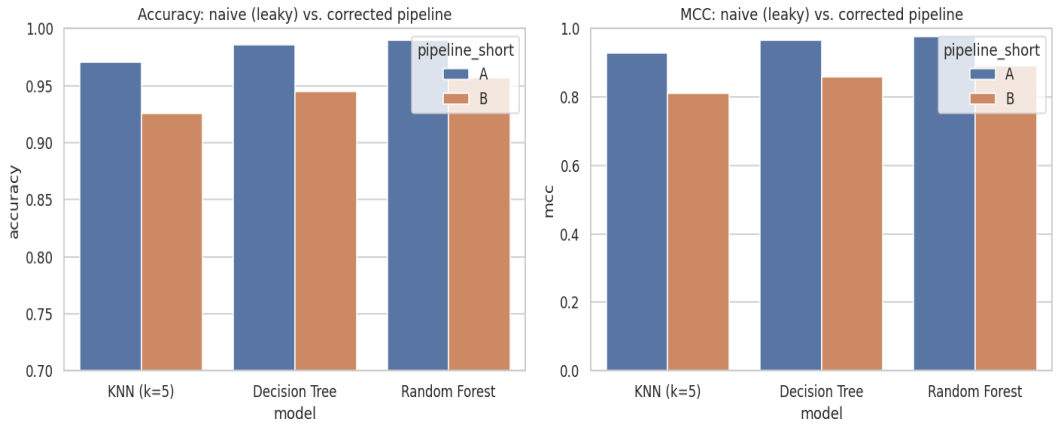


*Figure 1. Accuracy and MCC, naive (leaky) vs. corrected pipeline, per model. The drop is consistent across all three models and both metrics.*

## Cross-validated results (corrected pipeline, deduplicated data)

| Model | Mean MCC (5-fold CV) | Std. Dev. |
|---|---|---|
| KNN (k=5) | 0.8126 | ± 0.0057 |
| Decision Tree | 0.8663 | ± 0.0060 |
| Random Forest | 0.8978 | ± 0.0048 |

Cross-validation confirms the single-split results are stable (low variance across folds): Random Forest remains the strongest model, and KNN is the most leakage-sensitive and lowest-performing once duplicates are removed.

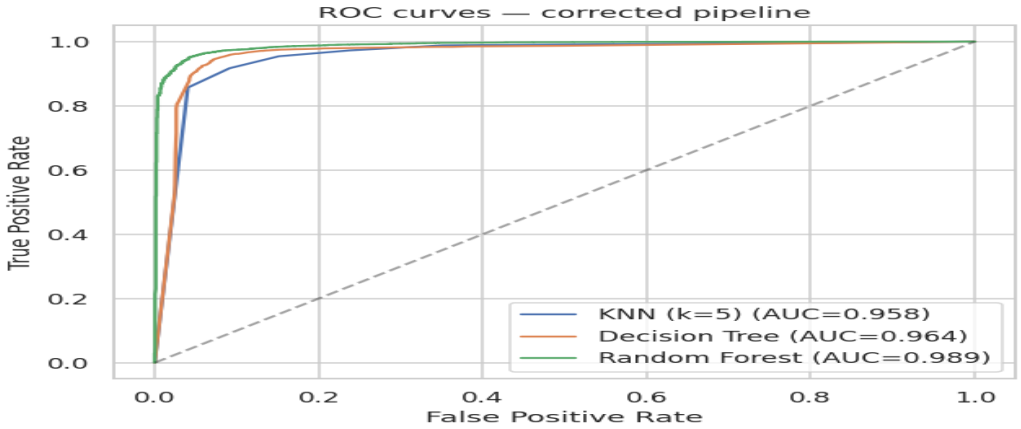


*Figure 2. ROC curves, corrected pipeline. All three models substantially outperform random guessing; Random Forest dominates across the full threshold range.*

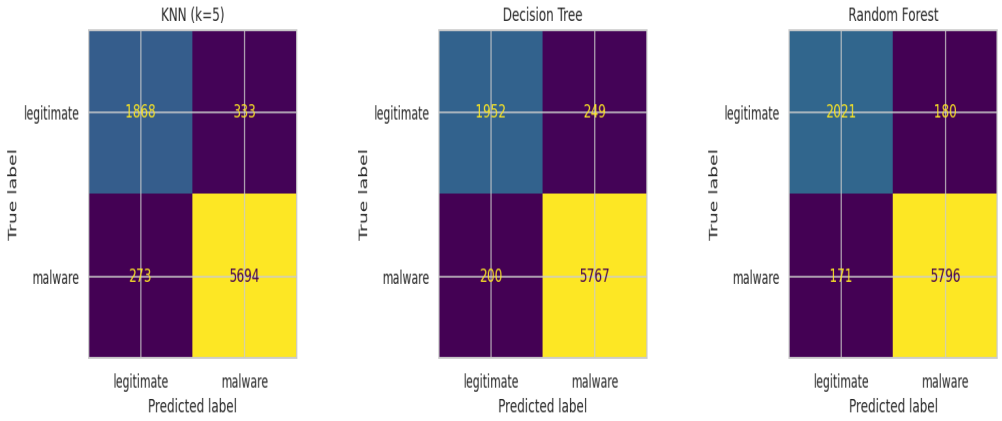


*Figure 3. Confusion matrices, corrected pipeline. Random Forest shows the best balance of False Positives and False Negatives among the three models.*

# 6. Error Analysis

Using the corrected-pipeline Random Forest model (our strongest, most reliable estimator), we examined the test-set errors directly rather than only summary metrics.

## Patterns in the errors

False Negatives (malware the model missed) tend to have header values closer to the “legitimate” class's typical profile — conventional linker versions, typical stack reserve sizes. These are plausibly malware samples built with standard, unmodified toolchains, giving them header metadata indistinguishable from benign software. This is an expected, structurally important weakness: a header-only model cannot reliably detect malware that does not look unusual at the header level, including malware that deliberately mimics legitimate build signatures to evade exactly this kind of detector.

## Cybersecurity implications

- **False Negatives are the more dangerous error class here**: an undetected piece of malware proceeds to execute. A header-only static model should realistically be one signal feeding a layered defense (alongside behavioral/dynamic analysis, signature matching, reputation scoring), not a sole gate.
- **False Positives** cost analyst time and can erode trust in the tool (alert fatigue), but are comparatively recoverable.
- Given this asymmetry, we would recommend tuning the classification threshold to favor recall (reflected in our Fβ=2 metric above) over the default 0.5 cutoff — accepting more false alarms in exchange for fewer missed detections, a standard SOC (Security Operations Center) trade-off.

## False Positive / False Negative trade-off

The confusion matrices in Figure 3 let us read this trade-off directly per model: Random Forest gives the best balance of both error types among the three models tested, which is why we treat it as the primary model for error analysis and for the deployment recommendation in Section 7.

# 7. Conclusions

## Key findings

- We reproduced the source's claimed ~97–99% accuracy almost exactly when following its exact (un-deduplicated, unscaled) methodology, confirming our replication is faithful.
- 76% of the dataset's rows are exact duplicates; deduplicating before splitting drops accuracy by 3–9 points and MCC by 0.06–0.12 depending on the model, with KNN affected most severely.
- The source's reported “F1” metric is mathematically identical to accuracy for this binary classification task, and therefore does not measure what its name implies.
- Feature scaling (absent in the source) measurably improves KNN; decoding the `DllCharacteristics` bitmask into individual flags yields more interpretable, comparably predictive features versus the raw integer.

- 5-fold cross-validation confirms the corrected results are stable; error analysis shows the dominant practical risk is False Negatives — malware built with standard toolchains that “looks” legitimate at the header level.

## Lessons learned

Headline accuracy figures in published ML-for-security tutorials should never be taken at face value without checking the evaluation procedure for leakage, especially when the dataset is small, derived from a narrow corpus, or (as here) contains substantial exact duplication. A single un-seeded, non-stratified hold-out split is an especially fragile basis for a strong claim. Metric names should be verified against their actual mathematical definitions, not assumed from the label given by the author.

## Strengths and weaknesses of the proposed solution

**Strengths:** the core hypothesis — static PE header metadata carries real, non-trivial signal for malware classification — survives correction (Random Forest still reaches ~95.7% accuracy / MCC ~0.89 on leakage-free, cross-validated data, from only 8 cheap, easy-to-extract integer features). The dataset is freely available and the modeling code, once patched for current scikit-learn, is simple and runs quickly.

**Weaknesses:** evaluation methodology (leakage, mislabeled metric, no seed, no cross-validation, no imbalance handling), near-total absence of feature engineering, no discussion of generalization beyond this specific corpus, and code that does not run as published on either a current scikit-learn install or (for the data generator) at all.

## Suggestions for future improvements

- Deduplicate before any train/test split, and always stratify and fix a random seed.
- Report precision/recall/MCC alongside accuracy, and verify any named metric against its actual formula.
- Add behavioral and content-based features (entropy, imports, signatures) rather than relying solely on header metadata, to reduce sensitivity to toolchain artifacts in a specific corpus.
- Validate on a held-out, separately-sourced malware corpus to test true generalization, rather than only a random split of the same collection.

## 8. Executive Summary

**Problem.** Detecting Windows PE malware from static header metadata alone, without executing the file.

**Source.** “Malware Detection Using Machine Learning” (emr4h, GitHub) trains KNN, Decision Tree, and Random Forest on 8 raw PE-header fields from a 137,444-row dataset, reporting 97.6–99.1% “success rate.”

**Dataset.** `MalwareDataSet.csv` — 96,526 legitimate / 40,918 malware samples (per the source's own labeling code, despite the misleadingly-named `legitimate` column actually encoding 1=malware).

**Methodology.** We reproduced the source's exact pipeline, then re-ran it correctly: deduplicating before splitting, stratifying the split, fixing a random seed, scaling features for the distance-based model, and evaluating with a fuller metric suite (precision, recall, F1, Fβ, MCC, ROC-AUC) plus 5-fold cross-validation.

**Main findings of our reproduction study.** We confirmed the author's reported numbers are real for their stated evaluation procedure, but that procedure leaks information: 76% of the dataset is exact duplicate rows, and splitting before deduplication lets near-identical rows appear in both train and test. Correcting this drops accuracy by 3–9 points and MCC substantially (e.g. Random Forest MCC 0.976 → 0.891 single-split, 0.898 cross-validated) across every model tested.

**Were the author's claims supported?** Partially. The underlying hypothesis — PE header metadata carries genuine signal for malware detection — holds up under correction. The specific ~99% accuracy figure as a measure of real-world generalization does not.

**Most important insights.** (1) Exact-duplicate rows are a serious, easily-overlooked leakage vector, and disproportionately affect one class here, which also flips the apparent class balance. (2) A metric's name should always be checked against its actual formula — micro-averaged F1 is accuracy in disguise for binary tasks. (3) Even simple, raw header features carry real signal once evaluated honestly, which is itself a useful finding for lightweight, low-cost malware triage.

**Would we recommend this approach for similar problems?** The general strategy (lightweight, header-only static classification as a fast first-pass filter) is reasonable and worth building on, but the source's *specific implementation and evaluation* should not be reused as-is; the deduplication, stratification, scaling, and metric-selection fixes described in this report should be applied first, and richer features (entropy, imports, signatures) added before any operational use.

**Final conclusion.** The project's core technical idea is sound and survives rigorous re-evaluation at a respectable (though lower) performance level, but its headline claim is an artifact of an inadequate evaluation methodology rather than a trustworthy estimate of real-world detection performance.